

Name:

Class:

Date:

Lab 15: Mini Create Task

Python Lab · Section 3.13 Procedures + Create Task practice



Goal

Build a small program that meets every Create Task requirement, then practice the written responses.

Warm-up: Predict

```
def count_long(words, n):  
    count = 0  
    for w in words:  
        if len(w) > n:  
            count += 1  
    return count
```

Point to the sequencing, selection, and iteration. Which parameter affects the result?

AP Connection

- Requirements: input, a list, a procedure with a parameter, an algorithm with sequencing + selection + iteration, a call to the procedure, and output.
- You'll answer written-response prompts about your code on exam day.

Build It

Step 1. Pick a mini idea: quiz game, playlist picker, workout planner, or grade tracker.

Step 2. Store data in a LIST that your program uses (it should be hard to write without the list).

```
workouts = ["push-ups", "squats", "plank", "lunges"]
```

Step 3. Write a PROCEDURE with a parameter whose algorithm uses sequencing, selection, and iteration.

```
def plan(level):  
    result = []  
    for w in workouts:  
        if level == "hard" or w != "plank":  
            result.append(w)  
    return result
```

Step 4. Get INPUT, CALL the procedure (twice, with different arguments), and produce OUTPUT.

```
choice = input("easy or hard? ")  
print(plan(choice))
```

Step 5. Take screenshots of (1) your list being used and (2) your procedure and where you call it. These are like your Personalized Project Reference.

Name: _____

Class: _____

Date: _____

Lab 15: Mini Create Task

Build It (continued) · Level Up

Level Up Challenges

Mild Comment every part to show where each requirement is met.

Medium Write a 1-minute video plan showing input, the program running, and output.

Spicy Answer the practice written-response prompts in the Create Task packet about YOUR mini program.

Python → AP Pseudocode

```
def plan(level):  
    result = []  
    for w in workouts:  
        if level == "hard" or w != "plank":  
            result.append(w)  
    return result
```

Write it in AP pseudocode.

Exit Ticket

Which requirement was hardest to meet? How did you meet it?
